

DOCUMENTACION DEL PROYECTO

1.1 Flyway

En un proyecto de arquitectura servidor-cliente (como una aplicación Java con Spring Boot y una base de datos), Flyway actúa como una herramienta de control de versiones de la base de datos. Su función principal es automatizar la evolución del esquema de la base de datos para que siempre esté sincronizado con el código del servidor. Funciones principales en el servidor:

Automatización de cambios: En lugar de ejecutar scripts SQL manualmente en cada entorno (desarrollo, test, producción), Flyway los ejecuta automáticamente al iniciar la aplicación.

Historial de versiones: Crea una tabla llamada `flyway_schema_history` donde registra qué scripts se han ejecutado, quién lo hizo y cuándo, evitando que un cambio se aplique dos veces.

Garantía de compatibilidad: Asegura que el servidor siempre encuentre las tablas y columnas exactas que necesita para funcionar, eliminando el error de "tabla no encontrada" tras una actualización de código.

Sincronización en equipo: Cuando se sube un cambio al repositorio (por ejemplo una nueva columna), Flyway detecta el nuevo script y actualiza la base de datos local automáticamente al arrancar el proyecto.

1.2 Nginx

¿Qué hace NGINX en un proyecto Servidor-Cliente?

En una arquitectura moderna, NGINX se sitúa frente a la aplicación (el backend de Spring Boot) actuando principalmente como **Proxy Inverso**.

Funciones principales

- **Proxy Inverso:** Recibe las peticiones de los clientes (navegador/app móvil) y las redirige al servidor Spring Boot. Esto protege al servidor, ya que el cliente nunca habla directamente con él, sino con NGINX.
- **Terminación SSL (HTTPS):** Es el lugar ideal para instalar tus certificados de seguridad. NGINX recibe la conexión cifrada (HTTPS), la descifra y le pasa la petición "limpia" al servidor por la red interna, ahorrándole trabajo de CPU a Java.

- **Balanceo de Carga:** Si en el futuro se escala un proyecto con, por ejemplo, 3 instancias del servidor ejecutándose en IntelliJ o en la nube, NGINX reparte el tráfico entre ellas para que ninguna se sature.
- **Servicio de Contenido Estático:** NGINX es extremadamente rápido entregando archivos que no cambian (HTML, CSS, JS, imágenes). Se puede configurar NGINX para que entregue el "Frontend" directamente, dejando que Spring Boot se encargue solo de la lógica y la base de datos (API).

Conexión con WebFlux

Dado que se usa un proyecto **WebFlux**, NGINX es un aliado crítico:

- **Gestión de conexiones:** WebFlux permite miles de conexiones simultáneas. NGINX está diseñado exactamente para lo mismo, por lo que pueden manejar picos de tráfico masivos de forma eficiente sin bloquearse.
- **Soporte de WebSockets:** Si se usa WebFlux para streaming de datos en tiempo real, NGINX puede mantener esas conexiones abiertas de forma persistente.

Integración en IntelliJ

Aunque NGINX suele ejecutarse fuera del IDE (en Docker o en el sistema operativo), IntelliJ ayuda a gestionarlo:

- **Configuración de archivos:** El plugin de NGINX en IntelliJ ofrece resaltado de sintaxis para los archivos .conf.
- **Pruebas locales:** Se puede configurar NGINX para que apunte a localhost:8080 (donde corre Spring Boot) y así probar el flujo de seguridad y rutas antes de subirlo a producción.

1.3 Docker

¿Qué aporta Docker a la infraestructura?

- **Aislamiento y Consistencia:** Garantiza que el servidor funcione exactamente igual en el equipo de desarrollo propio que en el de cualquier otro desarrollador o en producción, eliminando el problema de "en mi máquina funciona".
- **Gestión Multi-contenedor:** Permite que el backend (Spring Boot), la base de datos (ya sea Postgres o MySQL como es el caso) y el proxy (NGINX) funcionen como un ecosistema unido mediante una red virtual interna.
- **Portabilidad:** Todo el entorno del servidor se define en código (Dockerfile y docker-compose.yml), lo que facilita mover el proyecto completo entre diferentes nubes o servidores físicos.

Integración con las herramientas usadas en el proyecto

- **Con Flyway:** Se puede usar un contenedor temporal de Flyway que se ejecute justo antes de que el backend arranque. Esto asegura que la base de datos esté migrada y lista sin necesidad de tener Flyway instalado localmente.
- **Con NGINX:** Docker permite que NGINX actúe como la única "puerta de entrada" (puerto 80/443), redirigiendo el tráfico internamente hacia el contenedor de Spring Boot (puerto 8080) sin exponer este último directamente a Internet.
- **Persistencia de datos:** Mediante el uso de **volúmenes**, aseguras que los datos de la base de datos y los registros (*logs*) de NGINX no se pierdan aunque los contenedores se detengan o reinicien.

Conceptos Clave

- **Docker Compose:** Es la herramienta que orquestará el encendido de los servicios en el orden correcto (Base de datos -> Flyway -> Backend -> NGINX).
- **Redes Internas:** Docker crea una red privada donde los contenedores se comunican por sus nombres de servicio (ej: `http://backend:8080`), lo cual es más seguro que usar IPs públicas.
- **Imágenes Ligeras:** Se pueden usar versiones "Alpine" para NGINX y el Java JRE para que el servidor ocupe el mínimo espacio posible y arranque en segundos.

1.4 SpringBoot

Su función principal es **simplificar el desarrollo** de aplicaciones Java eliminando la configuración manual pesada. Se encarga de recibir las peticiones que le envía NGINX, procesar la lógica de negocio y hablar con la base de datos.

Puntos clave:

- **Auto-configuración:** Spring Boot "adivina" qué se necesita. Si detecta el driver de Postgres/MySQL y Flyway en un proyecto, configura la conexión automáticamente sin que se tenga que escribir código extra.
- **Servidor Embebido:** No se necesita instalar un servidor externo (como Tomcat o Netty) por separado; Spring Boot lo lleva dentro. Al ejecutar el archivo `.jar`, el servidor arranca al instante.
- **Gestión de Dependencias (Starters):** Usa "starters" (como `spring-boot-starter-webflux`) que agrupan todas las librerías necesarias para una función específica, evitando conflictos de versiones.
- **Producción Lista (Actuator):** Incluye herramientas para vigilar la salud del servidor (memoria usada, estado de la base de datos, logs) de forma nativa.

Conexión con las otras herramientas

- **Con Flyway:** Spring Boot detecta los scripts en db/migration y ordena a Flyway que los ejecute **antes** de que el resto de la aplicación esté disponible.
- **Con WebFlux:** Al usar el stack reactivo, Spring Boot utiliza un servidor **Netty** (en lugar de Tomcat) para manejar miles de conexiones simultáneas con muy pocos recursos de memoria.
- **Con Docker:** Spring Boot está diseñado para ser "cloud-native". Se empaqueta fácilmente en un contenedor Docker, permitiendo que variables de entorno (como la contraseña de la BD) se pasen desde el archivo docker-compose.yml.

Notas de Arquitectura

- **Modelo de Capas:** El código se organiza en **Controladores** (reciben datos), **Servicios** (lógica de negocio) y **Repositorios** (acceso a datos).
- **Ecosistema:** Es el pegamento que permite que IntelliJ, Docker y NGINX trabajen en armonía para entregar la API al cliente final.

1.5 Thymeleaf

Su función principal es el **Server-Side Rendering (SSR)**. A diferencia de frameworks como React o Angular (donde el navegador construye la página), con Thymeleaf es **Spring Boot** quien construye el HTML final y se lo envía "listo para mostrar" al cliente.

Funciones principales

- **Natural Templates:** Es su característica estrella. Los archivos .html de Thymeleaf se pueden abrir en un navegador y se ven como una web normal (diseño estático), pero cuando Spring Boot los procesa, sustituye las partes marcadas con datos reales de la base de datos.
- **Atributos Dinámicos:** Utiliza atributos especiales (como th:text, th:if o th:each) para mostrar nombres de usuarios, ocultar botones si no hay permisos, o crear listas dinámicas de productos.
- **Fragmentos Reutilizables:** Permite crear piezas de código (como un menú de navegación o un pie de página) una sola vez y reutilizarlas en todas las páginas del proyecto (th:replace o th:insert).
- **Integración con Spring Security:** Facilita mostrar u ocultar contenido dependiendo de si el usuario ha iniciado sesión o qué rol tiene, directamente en el HTML.

Conexión con el stack actual

- **Con Spring Boot:** Se integra de forma nativa. Solo con poner los archivos en `src/main/resources/templates`, Spring Boot sabe dónde buscarlos y cómo renderizarlos.
- **Con NGINX:** NGINX recibe la petición del usuario, la pasa a Spring Boot, este procesa el HTML con Thymeleaf y se lo devuelve a NGINX para que llegue al navegador.
- **Con WebFlux:** Aunque Thymeleaf es tradicionalmente bloqueante, existe una versión compatible con **Spring WebFlux** que permite renderizar las páginas de forma reactiva (en "pedazos" o *data-driven*), lo que mantiene el rendimiento alto del servidor.

Notas :

- **Ubicación:** Las plantillas viven en `src/main/resources/templates` y los archivos estáticos (CSS/JS) en `src/main/resources/static`.
- **Ventaja clave:** Es ideal para proyectos donde el SEO es importante o donde no se quiere tener que pensar en la complejidad de gestionar un proyecto de Frontend por separado, ya que todo vive dentro del mismo proyecto Java en IntelliJ.

1.6 Spring Security y mySql

Su función principal es gestionar la **Autenticación** (¿quién eres?) y la **Autorización** (¿qué tienes permiso de hacer?).

Puntos clave:

- **Seguridad en Capas:** Protege tanto las rutas web (URLs) como los métodos internos de la lógica de negocio.
- **Integración con MySQL:** Permite configurar un `UserDetailsService` que consulta directamente las tablas de usuarios y roles (creadas previamente por **Flyway**) para validar credenciales.
- **Cifrado de Contraseñas:** Incluye herramientas como BCrypt para que nunca haya que guardar contraseñas en texto plano dentro de MySQL, cumpliendo con estándares de seguridad.
- **Protección contra Ataques:** Activa por defecto defensas contra vulnerabilidades comunes como **CSRF** (falsificación de peticiones) y **XSS** (inyección de scripts). En este proyecto CSRF se ha dejado desactivado por conveniencia y simplicidad.

Conexión con el resto del stack:

- **Con WebFlux:** Utiliza un modelo reactivo (SecurityWebFilterChain) que no bloquea los hilos de ejecución, manteniendo la alta disponibilidad del servidor.
- **Con Thymeleaf:** Ofrece etiquetas especiales (como sec:authorize) para ocultar botones o menús en el HTML si el usuario no tiene los permisos adecuados.
- **Con NGINX:** NGINX puede encargarse del cifrado "exterior" (HTTPS), mientras que Spring Security se encarga del control de acceso "interior" (quién entra a qué sección).

1.7 BootStraap

BootStraap permite una gestión más eficaz de las páginas html de un proyecto.

¿Como funciona?

- Se añade Bootstraap al proyecto a través de un enlace CDN o un fichero local.
- Se usan las **clases predefinidas** de Bootstraap en las HTML del proyecto para darle forma a los elementos, en este caso se ha hecho fundamentalmente por medio de la página maestra layout.html que sirve de base para todas las demás. Recurriendo luego solamente de forma puntual si alguna página necesita un arreglo especial.
- El framework aplica automáticamente los estilos, la paginación et la adaptación a la pantalla.

La ventaja, se escribe menos en el CSS y se obtiene un diseño perfecto en todas las páginas de forma inmediata.

Sus características principales son :

- **Responsitivo de oficio** : El conjunto se adapta a smartphones, tabletas y pantallas anchas.
- **Sistema de rejill** potente para organizar las páginas
- **Componentes directamente utilizables** : botones, tarjetas, menús, alertas, formularios...
- **Personalizable**: se pueden modificar colores, tamaños, espacios a través de variables.
- **Compatible con JavaScript** : algunos de sus component tienen comportamientos interactivos.

1.8 Tabla Comparativa

Herramienta	Categoría	Misión Principal	Relación Clave
-------------	-----------	------------------	----------------

Docker	Infraestructura Contenedorización		Aísla el servidor y la base de datos MySQL.
NGINX	Proxy Inverso	Puerta de entrada	Recibe HTTPS y deriva el tráfico a Spring.
Spring Security	Seguridad	Control de acceso	Valida usuarios contra la BD MySQL.
Spring Boot	Backend	Cerebro / Lógica	Coordina WebFlux, Security y JPA.
MySQL	Base de Datos	Almacenamiento	Guarda los datos que Flyway organiza.
Flyway	Versionado DB	Evolución de tablas	Crea las tablas de usuarios para Security.
Thymeleaf	Frontend (SSR)	Interfaz dinámica	Muestra contenido según el rol del usuario.
Bootstrap	Html	Construcción simplificada de las páginas	Permite la construcción y gestión de las HTML de forma automatizada

Flujo de login

(cómo viaja el dato desde Thymeleaf hasta MySQL pasando por Security)

Flujo de Login: Del Navegador a la Base de Datos

Supongamos que un usuario intenta entrar a una zona privada de la web:

1. **Entrada (NGINX + Docker):** El usuario escribe sus credenciales en un formulario de **Thymeleaf**. Al dar clic en "Login", la petición viaja por la red, atraviesa **Docker** y llega a **NGINX** (puerto 80/443), que la redirige internamente al contenedor de **Spring Boot** (puerto 8080).
2. **Interceptación (Spring Security):** Antes de que la petición llegue a cualquier "Controller", **Spring Security** la atrapa. Al detectar que es una ruta de WebFlux, activa su filtro reactivo para extraer el usuario y la contraseña.
3. **Cifrado y Validación (BCrypt):** Spring Security no busca la contraseña "tal cual". Usa un codificador (como **BCrypt**) para preparar la comparación, garantizando que los datos sensibles nunca viajen desprotegidos en la memoria del servidor.

4. **Consulta a los Datos (MySQL + Flyway):** El "Cerebro" de Spring le pide al **Repositorio** que busque al usuario en la base de datos **MySQL**. Aquí es clave que **Flyway** haya creado correctamente la tabla users al arrancar el proyecto; de lo contrario, este paso fallaría.

5. **Decisión de Acceso:**

- **Si coinciden:** Spring Security crea una "Sesión" o "Token" en memoria y permite que el flujo continúe.
- **Si fallan:** Corta la petición y redirige al usuario de vuelta al login con un mensaje de error.

Respuesta Visual (Thymeleaf): Una vez logueado, **Spring Boot** le entrega al usuario la página solicitada. **Thymeleaf** detecta mediante etiquetas (ej. sec:authorize) que el usuario ya es válido y "dibuja" en el HTML opciones nuevas, como el botón de "Cerrar Sesión" o el panel de administrador.

El flujo de seguridad es un **trabajo en equipo**: NGINX recibe, Spring Security valida, MySQL/Flyway proveen los datos y Thymeleaf personaliza la interfaz final. Todo esto ocurre de forma asíncrona gracias a **WebFlux**.

1.9 Resumen:

Informe de Arquitectura: Sistema Distribuido Multi-módulo

1. Resumen Ejecutivo

El proyecto **distributed-app** consiste en una plataforma web distribuida y escalable, estructurada en microservicios y módulos especializados. La arquitectura combina la potencia del ecosistema **Java (Spring)** para la lógica de negocio y la flexibilidad de **Python** para servicios específicos, todo ello orquestado bajo contenedores **Docker** y supervisado por estándares de calidad **SonarQube**.

2. Estructura de Módulos

El proyecto se organiza bajo una jerarquía multi-módulo que separa responsabilidades de forma clara:

- **backend (Java/Spring Boot):** Actúa como el núcleo de procesamiento REST, gestionando la lógica de negocio y la persistencia avanzada.
- **frontend (Java/Spring MVC + Thymeleaf):** Capa de presentación que renderiza la interfaz de usuario de forma dinámica y segura.
- **py1 (Python):** Microservicio especializado en el consumo y procesamiento de la **Pokémon API**.

- **py2 (Python):** Microservicio dedicado a operaciones **CRUD** directas sobre la base de datos **MySQL 8**.

3. Infraestructura y Flujo de Datos

3.1. Orquestación y Persistencia

La infraestructura se despliega mediante **Docker**, garantizando que el entorno de desarrollo sea idéntico al de producción.

- **MySQL 8:** Repositorio central de datos, cuya integridad y evolución de esquemas es gestionada automáticamente por **Flyway**.
- **SonarQube:** Integrado en el ciclo de vida para asegurar que cada módulo cumpla con los umbrales de calidad y seguridad de código.

3.2. Enrutamiento Global (Nginx como Reverse Proxy)

Nginx actúa como el punto único de entrada, ocultando la complejidad interna de la red y mejorando la seguridad. El esquema de ruteo configurado es el siguiente:

Ruta (URL) Destino (Módulo) Función

/	frontend	Interfaz de usuario (Spring MVC)
/backend/*	backend	API Principal y Lógica de Negocio
/api2/*	py1	Servicio Python - Pokémon API
/api3/*	py2	Servicio Python - CRUD MySQL

4. Capas de Servicio y Seguridad

1. **Seguridad (Spring Security):** Implementada para gestionar el control de acceso a los datos en **MySQL**, protegiendo las rutas sensibles y validando identidades antes de permitir el procesamiento de peticiones.
2. **Gestión de Errores:** El sistema cuenta con una configuración personalizada en Nginx y Spring para capturar excepciones, redirigiendo al usuario a páginas de error específicas (404, 500, etc.) diseñadas en **Thymeleaf**, mejorando así la experiencia de usuario (UX).
3. **Reactividad:** El uso de **Spring WebFlux** en los componentes Java permite que el sistema maneje un alto volumen de peticiones concurrentes sin bloquear los hilos del servidor.

5. Conclusión de Diseño

Esta arquitectura modular no solo facilita el mantenimiento independiente de cada servicio, sino que permite que tecnologías diversas (Java y Python) coexistan de forma armónica bajo un mismo nombre de dominio, protegidas por un firewall de aplicaciones y un sistema de migraciones de datos automatizado.

El uso de **Nginx como Reverse Proxy** y la estructura **multi-módulo** son estándares de la industria que hacen que la aplicación sea "escalable": si el tráfico de los microservicios de Python crece, podrías moverlos a otros servidores sin que el usuario note ningún cambio en las URLs.